

Kubernetes Interview & Troubleshooting Notes

Topics 12–18

Topics Covered

12. Types of Services in Kubernetes
 13. ClusterIP
 14. Application + Database — Which Service and Why?
 15. Pod YAML
 16. User + Authorization using RBAC
 17. HPA — Horizontal Pod Autoscaler
 18. ClusterIP Service YAML
 19. Real-Time Troubleshooting Scenarios
 20. Quick Interview Answers
-

12. Types of Services in Kubernetes

12.1 What is a Kubernetes Service?

A Kubernetes **Service** provides a stable network endpoint to access a group of Pods.

Pods are temporary. Their IP addresses can change when Pods are recreated.

For example:

Backend Pod 1

IP: 10.244.1.10

Backend Pod 2

IP: 10.244.2.15

If Backend Pod 1 crashes, Kubernetes may create another Pod:

New Backend Pod

IP: 10.244.3.20

The IP changed.

Instead of applications directly connecting to Pod IPs, we create a Service:

Frontend

|

v

backend-service

|

+-----> Backend Pod 1

|

+-----> Backend Pod 2

|

+-----> Backend Pod 3

The frontend always connects to:

backend-service

The Service handles routing traffic to the available Pods.

12.2 Why Do We Need Services?

Services provide:

- Stable IP address
- Stable DNS name
- Load balancing between Pods
- Service discovery
- Decoupling between applications and Pods

Without a Service:

Frontend ---> Pod IP

With a Service:

Frontend ---> Service ---> Pod

The second architecture is much more reliable.

12.3 Types of Kubernetes Services

The major Service types are:

1. ClusterIP
2. NodePort
3. LoadBalancer
4. ExternalName
5. Headless Service

12.4 ClusterIP

ClusterIP is the **default Service type**.

It exposes the application **inside the Kubernetes cluster**.

Example:

Frontend Pod

|

v

Backend Service

ClusterIP

|

+----> Backend Pod

|

+----> Backend Pod

It is commonly used for:

- Backend APIs
- Databases
- Internal microservices
- Redis
- Internal communication

Example:

apiVersion: v1

kind: Service

metadata:

name: backend-service

spec:

selector:

app: backend

ports:

- protocol: TCP

port: 80

targetPort: 8080

type: ClusterIP

Important

port

is the Service port.

targetPort

is the port where the application is running inside the Pod.

Example:

Client

|

| Port 80

v

Service

|

| targetPort 8080

v

Backend Container

12.5 NodePort

NodePort exposes a Service through a port on every Kubernetes Node.

Example:

Internet

|

v

Node IP:30080

|

v

NodePort Service

|

v

Backend Pods

NodePort normally uses the range:

30000 - 32767

Example:

apiVersion: v1

kind: Service

metadata:

name: backend-nodeport

spec:

selector:

app: backend

ports:

- port: 80

targetPort: 8080

nodePort: 30080

type: NodePort

You can access it using:

http://<Node-IP>:30080

When is NodePort useful?

Mostly:

- Development
- Testing
- Minikube
- Lab environments

It is generally not the preferred way to expose a production web application directly to users.

12.6 LoadBalancer

LoadBalancer exposes a Service using an external cloud load balancer.

Typical cloud environments:

- AWS
- Azure
- Google Cloud

Example:

Internet

|

v

Cloud Load Balancer

|

v

Kubernetes Service

|

+----> Pod

+----> Pod

+----> Pod

Example:

apiVersion: v1

kind: Service

metadata:

name: frontend-service

spec:

selector:

app: frontend

ports:

- port: 80

targetPort: 80

type: LoadBalancer

In a cloud environment, Kubernetes can provision an external load-balancing resource depending on the platform and its configuration.

12.7 ExternalName

ExternalName allows a Kubernetes Service to refer to an external DNS name.

Example:

apiVersion: v1

kind: Service

metadata:

name: external-db

spec:

type: ExternalName

externalName: database.example.com

The application can use:

external-db

instead of directly using the external hostname.

This can be useful when an application inside Kubernetes needs to access an external service.

12.8 Headless Service

A Headless Service does not receive a normal ClusterIP.

It is created using:

clusterIP: None

Example:

apiVersion: v1

kind: Service

metadata:

name: database

spec:

clusterIP: None

selector:

app: database

ports:

- port: 3306

Headless Services are commonly useful with:

- StatefulSets
- Databases
- Distributed systems
- Applications that need direct Pod discovery

12.9 Service Comparison

Service	Main Purpose
ClusterIP	Internal communication
NodePort	Expose through Node IP and port
LoadBalancer	External cloud load balancer
ExternalName	External DNS mapping
Headless	Direct Pod discovery

13. Explain ClusterIP

13.1 Definition

ClusterIP is the default Kubernetes Service type that provides a stable internal IP and DNS name for accessing a group of Pods inside the cluster.

Interview answer:

ClusterIP is the default Service type in Kubernetes. It exposes an application internally within the cluster and provides a stable IP and DNS name even when the underlying Pod IPs change.

13.2 Example

Suppose we have three backend Pods:

backend-pod-1

10.244.1.10

backend-pod-2

10.244.2.15

backend-pod-3

10.244.3.20

We create:

backend-service

The Service gets a ClusterIP:

10.96.100.50

Frontend does not need to know the individual Pod IPs.

It calls:

http://backend-service

The Service routes traffic to the backend Pods.

13.3 ClusterIP Architecture

Frontend Pod

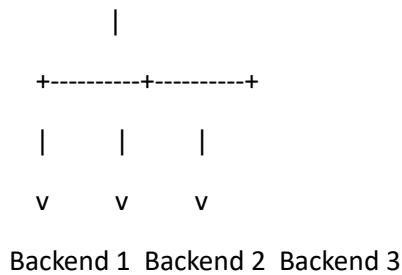
|

|

v

backend-service

ClusterIP



13.4 Why ClusterIP Instead of Pod IP?

Pod IP:

Temporary

ClusterIP:

Stable

Example:

Pod dies

|

v

New Pod created

|

v

New Pod IP

The Service remains the same.

Therefore:

Application ---> Service

is preferred over:

Application ---> Pod IP

13.5 Service DNS

Kubernetes provides DNS names for Services.

For example:

backend-service

Inside the same namespace, applications can generally use:

http://backend-service

The fully qualified DNS form can be:

backend-service.default.svc.cluster.local

where:

backend-service = Service name

default = Namespace

svc = Service DNS domain

cluster.local = Cluster domain

13.6 ClusterIP Troubleshooting Commands

Check Services:

kubectl get svc

Describe Service:

kubectl describe svc backend-service

Check endpoints:

kubectl get endpoints backend-service

Check EndpointSlices:

kubectl get endpointslice

Check Pod labels:

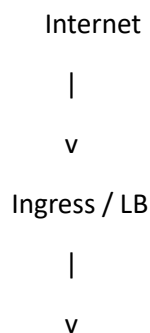
kubectl get pods --show-labels

14. Application + Database — Which Service and Why?

Interview Question

You have a frontend application, backend application and database. Which Service types will you use?

A common architecture is:



Frontend Service

|

v

Frontend Pods

|

v

Backend Service

ClusterIP

|

v

Backend Pods

|

v

MySQL Service

ClusterIP

|

v

MySQL Pods

14.1 Frontend

For external users, use:

Ingress

or:

LoadBalancer

depending on the architecture.

The frontend needs to be reachable by users.

14.2 Backend

Use:

ClusterIP

Why?

Because the backend normally only needs to be accessed by other applications inside the cluster.

Example:

Frontend ---> backend-service

14.3 Database

Use:

ClusterIP

Why?

The database should normally not be directly exposed to the Internet.

Example:

Backend

|

v

mysql-service

|

v

MySQL

14.4 Why Not Expose MySQL Using NodePort?

Bad design:

Internet

|

v

NodePort

|

v

MySQL

This unnecessarily exposes the database.

Better:

Internet

|

v
Frontend

|
v
Backend

|
v
MySQL

The database remains internal.

14.5 Interview Answer

For an application and database architecture, I would normally keep the backend and database behind ClusterIP Services because they require internal communication. For the user-facing application, I would use Ingress or a LoadBalancer depending on the environment. I would not expose the database directly to the Internet.

15. Write a Pod YAML

15.1 What is a Pod?

A Pod is the **smallest deployable unit in Kubernetes**.

A Pod can contain:

- One container
- Multiple closely related containers

Most applications commonly run one main application container per Pod.

15.2 Simple Pod YAML

apiVersion: v1

kind: Pod

metadata:

name: nginx-pod

labels:

app: nginx

spec:

containers:

- name: nginx-container

image: nginx:latest

ports:

- containerPort: 80

15.3 Explanation

apiVersion

apiVersion: v1

Specifies the Kubernetes API version.

kind

kind: Pod

Tells Kubernetes that we are creating a Pod.

metadata

metadata:

name: nginx-pod

Defines the Pod name.

labels

labels:

app: nginx

Labels identify the Pod.

Services use these labels to find Pods.

containers

containers:

Defines the containers inside the Pod.

image

image: nginx:latest

Specifies the container image.

containerPort

containerPort: 80

Documents the port the application container is expected to listen on.

It does not by itself expose the Pod outside the cluster.

15.4 Create Pod

Save as:

pod.yaml

Run:

kubectl apply -f pod.yaml

Check:

kubectl get pods

Detailed information:

kubectl describe pod nginx-pod

Logs:

kubectl logs nginx-pod

15.5 Pod Lifecycle

A Pod can go through states such as:

Pending

|

v

Running

|

+-----> Succeeded

|

+-----> Failed

You may also encounter conditions such as:

CrashLoopBackOff

ImagePullBackOff

ErrImagePull

Pending

These are important troubleshooting states.

15.6 Important Interview Point

In production, applications are usually managed through higher-level controllers such as:

Deployment

StatefulSet

DaemonSet

Job

CronJob

Rather than manually creating individual Pods.

16. Add User + Authorization — RBAC

16.1 What is RBAC?

RBAC stands for:

Role-Based Access Control

RBAC controls:

WHO

|

v

CAN DO WHAT

|

v

ON WHICH RESOURCE

Example:

Developer

|

+----> get Pods

+----> list Pods

+----> view logs

But:

Developer

|

X----> delete namespace

16.2 Why RBAC?

Without proper authorization, every user could potentially have excessive access.

With RBAC, we follow the principle of:

Least Privilege

Give users only the permissions they need.

16.3 Main RBAC Components

Component	Purpose
User	Identity used by Kubernetes authentication
ServiceAccount	Identity for workloads
Role	Permissions within a namespace
RoleBinding	Assigns a Role
ClusterRole	Cluster-level or reusable permissions
ClusterRoleBinding	Assigns a ClusterRole cluster-wide

16.4 Important Difference

Role

Role is namespace-scoped.

Example:

Role in namespace "dev"

It controls resources within that namespace.

ClusterRole

ClusterRole is not limited to a single namespace in its definition and can provide cluster-scoped permissions.

16.5 Example: Developer Can Read Pods

First create a namespace:

```
kubectl create namespace dev
```

Step 1 — Create ServiceAccount

```
apiVersion: v1
```

```
kind: ServiceAccount
```

```
metadata:
```

```
  name: dev-user
```

```
  namespace: dev
```

Apply:

```
kubectl apply -f serviceaccount.yaml
```

Step 2 — Create Role

Allow the identity to:

```
get
```

```
list
```

```
watch
```

```
Pods.
```

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: Role
```

```
metadata:
```

name: pod-reader

namespace: dev

rules:

- apiGroups:

- ""

resources:

- pods

verbs:

- get

- list

- watch

Step 3 — Create RoleBinding

apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding

metadata:

name: pod-reader-binding

namespace: dev

subjects:

- kind: ServiceAccount

name: dev-user

namespace: dev

roleRef:

kind: Role

name: pod-reader

apiGroup: rbac.authorization.k8s.io

Apply:

```
kubectl apply -f role.yaml
```

```
kubectl apply -f rolebinding.yaml
```

16.6 Test Authorization

Check whether the ServiceAccount can list Pods:

```
kubectl auth can-i list pods \
```

```
--as=system:serviceaccount:dev:dev-user \
```

```
-n dev
```

Expected:

yes

Check whether it can delete Deployments:

```
kubectl auth can-i delete deployments \
```

```
--as=system:serviceaccount:dev:dev-user \
```

```
-n dev
```

Expected:

no

This demonstrates least privilege.

16.7 RBAC Flow

Identity

|

v

ServiceAccount

|

v

RoleBinding

|

v

Role

|

v

Permissions

For example:

dev-user

|

v

pod-reader-binding

|

v

pod-reader

|

+---- get pods

+---- list pods

+---- watch pods

16.8 Interview Answer

RBAC is Kubernetes authorization based on roles. It allows us to define what actions a user or ServiceAccount can perform on specific Kubernetes resources. Role and RoleBinding are namespace-scoped, while ClusterRole and ClusterRoleBinding can provide broader permissions.

17. What is HPA?

17.1 Definition

HPA stands for:

Horizontal Pod Autoscaler

HPA automatically increases or decreases the number of Pods based on metrics.

For example:

Low traffic

|

v

2 Pods

During high traffic:

High traffic

|

v

8 Pods

When traffic decreases:

Low traffic

|

v

2 Pods

17.2 Why HPA?

Imagine a Java backend application.

Normal traffic:

CPU = 30%

Pods = 2

Traffic increases:

CPU = 80%

HPA can increase:

2 Pods

|

v

4 Pods

|

v

6 Pods

This helps the application handle increased workload.

17.3 HPA Architecture

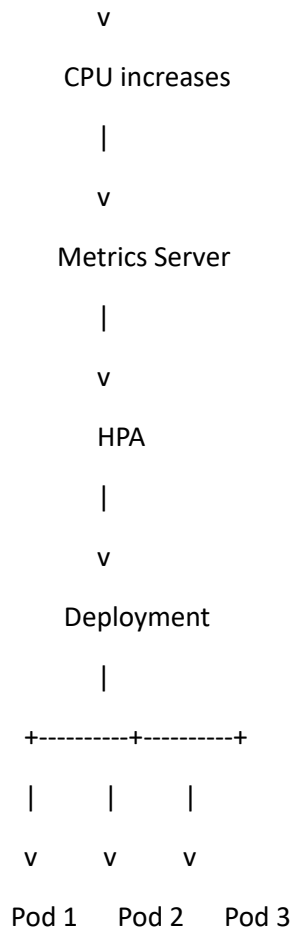
Application Traffic

|

v

Application

|



17.4 HPA Requirements

For CPU/memory-based HPA, you generally need:

1. A scalable workload such as a Deployment.
 2. Resource requests configured for the containers.
 3. Metrics available through the cluster's metrics pipeline, commonly Metrics Server for basic resource metrics.
-

17.5 Resource Requests

Example:

resources:

requests:

cpu: "100m"

memory: "128Mi"

limits:

cpu: "500m"

memory: "256Mi"

The CPU request is important when using CPU utilization as the HPA target.

17.6 HPA YAML

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: backend-hpa

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: backend

minReplicas: 2

maxReplicas: 10

metrics:

- type: Resource

resource:

name: cpu

target:

type: Utilization

averageUtilization: 70

17.7 Explanation

scaleTargetRef

scaleTargetRef:

Defines which workload HPA should scale.

Here:

Deployment/backend

minReplicas

minReplicas: 2

The application should normally have at least two replicas according to this HPA configuration.

maxReplicas

maxReplicas: 10

HPA should not scale beyond ten replicas under this configuration.

averageUtilization

averageUtilization: 70

The target average CPU utilization is 70% of the configured CPU requests.

17.8 Check HPA

kubectl get hpa

Detailed:

kubectl describe hpa backend-hpa

Check metrics:

kubectl top pods

Check nodes:

kubectl top nodes

17.9 HPA Troubleshooting

Problem

HPA exists but Pods are not scaling.

Check:

```
kubectl get hpa
```

Then:

```
kubectl describe hpa backend-hpa
```

Check metrics:

```
kubectl top pods
```

Check Deployment:

```
kubectl get deployment
```

Check resource requests:

```
kubectl get deployment backend -o yaml
```

Common Problems

Problem 1 — Metrics unavailable

You may see:

```
unknown
```

Check:

```
kubectl top pods
```

If metrics are unavailable, inspect the metrics pipeline/Metrics Server.

Problem 2 — Resource requests missing

Example:

```
resources:
```

```
  requests:
```

```
    cpu: "100m"
```

should be configured when using CPU utilization targets.

Problem 3 — Wrong target

Check:

```
scaleTargetRef:
```

Make sure the Deployment name is correct.

17.10 Interview Answer

HPA automatically adjusts the number of Pods in a workload based on metrics such as CPU or memory utilization. It helps applications handle changing traffic while avoiding unnecessary replicas during low traffic.

18. Service YAML — ClusterIP

This is one of the most important YAML files for Kubernetes interviews.

18.1 Complete Service YAML

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: backend-service
```

```
spec:
```

```
  selector:
```

```
    app: backend
```

```
  ports:
```

```
    - protocol: TCP
```

```
      port: 80
```

```
      targetPort: 8080
```

```
  type: ClusterIP
```

18.2 Line-by-Line Explanation

apiVersion

```
apiVersion: v1
```

Service belongs to the Kubernetes core API group.

kind

```
kind: Service
```

We are creating a Service.

metadata.name

metadata:

name: backend-service

The Service name is:

backend-service

Other applications can use this name for service discovery.

selector

selector:

app: backend

This is extremely important.

The Service looks for Pods with:

labels:

app: backend

For example:

metadata:

labels:

app: backend

If the labels do not match, the Service will not have the expected endpoints.

18.3 port vs targetPort

This is a very common interview question.

ports:

- port: 80

targetPort: 8080

Meaning:

Client

|

| Port 80

v

Service

|

| targetPort 8080

v

Application Container

port

The port exposed by the Service.

80

targetPort

The port on the Pod/container receiving the traffic.

8080

18.4 Complete Backend Example

Deployment:

apiVersion: apps/v1

kind: Deployment

metadata:

name: backend

spec:

replicas: 3

selector:

matchLabels:

app: backend

template:

metadata:

labels:

app: backend

spec:

containers:

- name: backend

image: nginx

ports:

- containerPort: 8080

Service:

apiVersion: v1

kind: Service

metadata:

name: backend-service

spec:

selector:

app: backend

ports:

- port: 80

targetPort: 8080

type: ClusterIP

18.5 How the Service Finds Pods

Service:

selector:

app: backend

Pod:

labels:

app: backend

Because they match:

Service

```
|  
| selector: app=backend  
|  
+-----> Pod app=backend  
|  
+-----> Pod app=backend  
|  
+-----> Pod app=backend
```

18.6 Check Endpoints

Run:

kubectl get endpoints backend-service

or:

kubectl get endpointslice

You should see addresses corresponding to matching backend Pods.

If you see:

<none>

investigate the selector and Pod labels first.

18.7 Test the Service

Create a temporary test Pod:

```
kubectl run test-pod \  
  --image=busybox:1.36 \  
  -it \  
  --rm \  
  --restart=Never \  
  -- sh
```


Inside the Pod:

```
wget -qO- http://backend-service
```

If the backend is correctly configured and listening, you should receive a response.

18.8 Port Forwarding

You can also test a Service from your local machine:

```
kubectl port-forward svc/backend-service 8080:80
```

Then access:

```
http://localhost:8080
```

This is useful for debugging.

19. Real-Time Kubernetes Troubleshooting

These scenarios are particularly useful in DevOps interviews.

Scenario 1 — Service Has No Endpoints

Interview Question

My Service is running, but `kubectl get endpoints` shows `<none>`. What will you check?

Answer

First check Service:

```
kubectl describe svc backend-service
```

Check selector:

```
Selector: app=backend
```

Then check Pod labels:

```
kubectl get pods --show-labels
```

If Pods have:

```
app=frontend
```

but Service expects:

```
app=backend
```

the Service will not select them.

Fix

Make the labels and selector match.

Scenario 2 — Service Exists but Application Is Not Reachable

Check:

```
kubectl get svc
```

Then:

```
kubectl describe svc backend-service
```

Then:

```
kubectl get endpoints backend-service
```

If endpoints exist, check the target port.

For example:

```
port: 80
```

```
targetPort: 8080
```

Make sure the application is actually listening on the expected port.

Scenario 3 — DNS Is Not Working

From a temporary Pod:

```
kubectl run dns-test \
  --image=busybox:1.36 \
  -it \
  --rm \
  --restart=Never \
  -- sh
```

Try:

```
nslookup backend-service
```

If DNS resolution fails, inspect CoreDNS:

```
kubectl get pods -n kube-system
```

Depending on the cluster setup, inspect CoreDNS logs:

```
kubectl logs -n kube-system deployment/coredns
```

Scenario 4 — HPA Is Not Scaling

Check:

kubectl get hpa

Then:

kubectl describe hpa backend-hpa

Check metrics:

kubectl top pods

Check resource requests.

Check:

scaleTargetRef

Make sure it points to the correct Deployment.

Scenario 5 — User Cannot Access Kubernetes Resource

Check authorization:

```
kubectl auth can-i get pods \
  --as=system:serviceaccount:dev:dev-user \
  -n dev
```

If output is:

no

check:

Role

RoleBinding

Subject

Namespace

The most common mistake is an incorrect RoleBinding or namespace.

20. Important Interview Questions

Q1. What is a Service?

Answer:

A Service provides a stable network endpoint for accessing a group of Pods and enables service discovery and traffic routing.

Q2. What is ClusterIP?

Answer:

ClusterIP is the default Service type. It provides an internal stable IP and DNS name for communication within the Kubernetes cluster.

Q3. What is the difference between port and targetPort?

Answer:

port is the port exposed by the Service, while targetPort is the port on the Pod/container where the application receives traffic.

Q4. What happens if a Pod's IP changes?

Answer:

Applications should not depend directly on the Pod IP. They communicate through a Service, whose stable IP and DNS name remain available while Kubernetes updates the underlying endpoints.

Q5. Which Service would you use for a database?

Answer:

Normally ClusterIP, because the database should remain accessible only to applications inside the cluster. For production databases, the database may instead be managed outside the cluster, such as a cloud-managed database.

Q6. Which Service would you use to expose a frontend?

Answer:

In a production Kubernetes environment, I would commonly use Ingress/Gateway with an appropriate LoadBalancer entry point, or a LoadBalancer Service depending on the architecture. For simple testing, NodePort can also be used.

Q7. What is RBAC?

Answer:

RBAC stands for Role-Based Access Control. It controls which users or ServiceAccounts can perform which actions on Kubernetes resources.

Q8. What is the difference between Role and ClusterRole?

Answer:

A Role defines permissions within a namespace. A ClusterRole can define permissions at cluster scope and can also be bound into a namespace through a RoleBinding.

Q9. What is HPA?

Answer:

HPA automatically increases or decreases the number of Pods based on resource or custom metrics such as CPU utilization.

Q10. Why is HPA not scaling?

Check:

`kubectl get hpa`

`kubectl describe hpa`

`kubectl top pods`

Then verify:

- Metrics availability
 - CPU/memory requests
 - HPA target
 - Deployment reference
 - Min/max replicas
-

21. Most Important Commands

Services

`kubectl get svc`

`kubectl describe svc <service-name>`

`kubectl get endpoints <service-name>`

`kubectl get endpointslice`

Pods

`kubectl get pods`

`kubectl get pods --show-labels`

`kubectl describe pod <pod-name>`

`kubectl logs <pod-name>`

`kubectl exec -it <pod-name> -- sh`

Deployments

kubectl get deployment

kubectl describe deployment <deployment-name>

RBAC

kubectl get role

kubectl get rolebinding

kubectl auth can-i get pods

HPA

kubectl get hpa

kubectl describe hpa <hpa-name>

kubectl top pods

kubectl top nodes

DNS Testing

kubectl run dns-test \

--image=busybox:1.36 \

-it \

--rm \

--restart=Never \

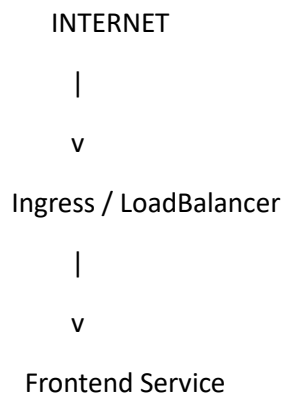
-- sh

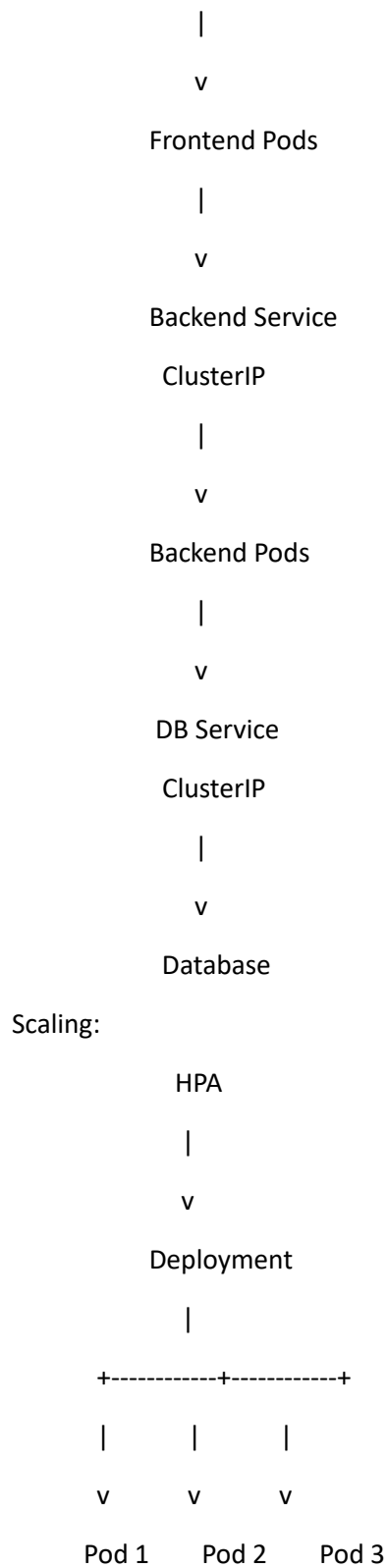
Then:

nslookup backend-service

22. Final Architecture to Remember

For a typical three-tier application:





Authorization:

User / ServiceAccount



RoleBinding

|

v

Role

|

v

Kubernetes API

23. One-Minute Interview Revision

Services

ClusterIP = Internal

NodePort = Node IP + Port

LoadBalancer = External Cloud Load Balancer

ExternalName = External DNS

Headless = Direct Pod Discovery

ClusterIP

Stable internal IP + DNS

Application + Database

Frontend -> Ingress/LB

Backend -> ClusterIP

Database -> ClusterIP

Pod

Smallest deployable Kubernetes unit

RBAC

WHO can DO WHAT on WHICH RESOURCE

HPA

Automatically changes Pod count based on metrics

Service YAML

Remember:

selector:

app: backend

ports:

- port: 80

targetPort: 8080

type: ClusterIP

The most important concept is:

Service selector

|

v

Pod labels

|

v

Endpoints

|

v

Pod traffic

If you understand this flow, you can troubleshoot many Kubernetes Service issues in a real DevOps interview.